Pintle Aero: Overview

AI-Powered Requirements Traceability & Verification Agent for Aerospace Engineering

Pintle Aero is my personal passion project. It is an engineering review assistant that I created with the help of AI.

Pintle Aero ingests requirements documents, test procedures, and test reports, and then automatically extracts structured data, suggests traceability links, detects verification gaps and cross-document inconsistencies, and produces review-ready exports.

It answers the four questions every systems engineer needs answered before a design review:

1. **What requirements are not verified?**
2. **What evidence supports each requirement?**
3. **Where do documents disagree?**
4. **Can I export something useful for a design review today?**

> **Note:** This is a portfolio showcase repository. The source code is maintained in a private repository. For a live demo, please contact me.

The Problem

Engineering teams in aerospace, defense, and other regulated industries maintain requirements, test procedures, and test reports across disconnected Word, PDF, Excel, and text files. Traceability is manually assembled in spreadsheets: it is time-consuming, error-prone, and often incomplete.

Critical issues hide in plain sight:

- A structural requirement that no test report covers
- A test report marked "PASS" when the measured value actually exceeds the threshold
- A requirement specifying limits in pounds-force while the test report measures in Newtons
- Two subsystem specs defining contradictory acceptance criteria for the same parameter

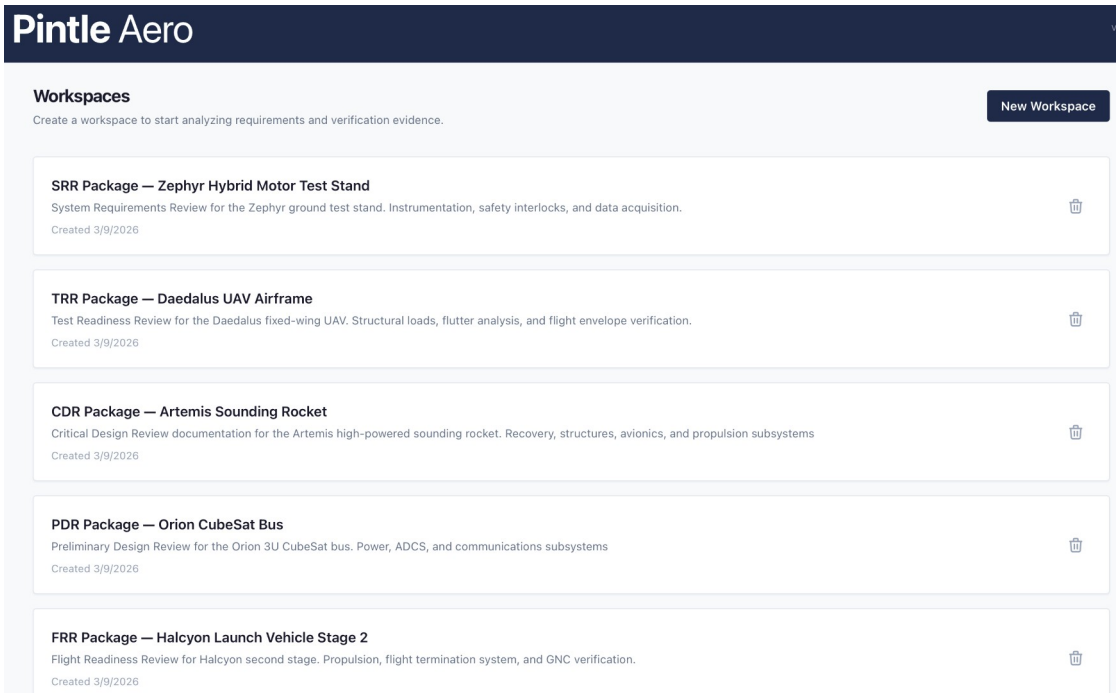
These problems are typically discovered during high-pressure design reviews, when they are expensive to fix and embarrassing to explain. Pintle Aero catches them early. The idea was born from firsthand experience: during my NASA Student Launch project at Purdue, incomplete traceability was a recurring pain point at every design review. We spent hours manually cross-referencing requirements against test reports in spreadsheets, and critical gaps still slipped through. I kept thinking: there has to be a better way to do this. This is an attempt to find a better way!

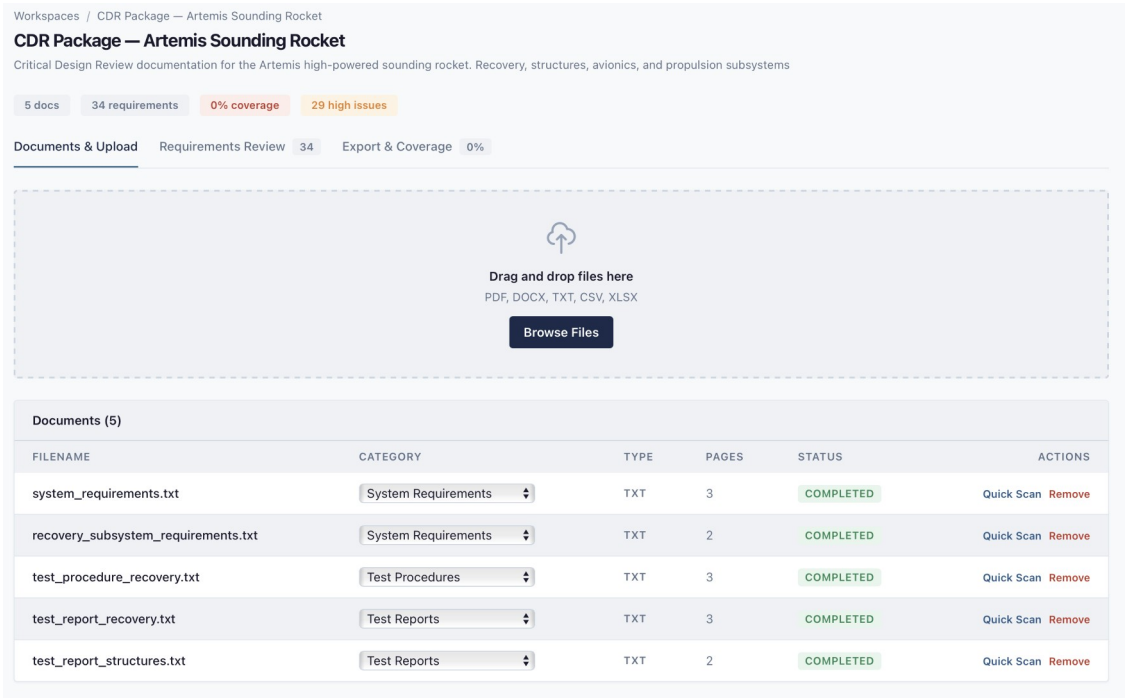
How It Works

```
┌──────────────────────────────────────────────────────────────┐
│                        UPLOAD DOCUMENTS                      │
│          Requirements Specs · Test Procedures · Test Reports │
│                   PDF · DOCX · XLSX · CSV · TXT              │
└──────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────┐
│                    HYBRID EXTRACTION PIPELINE                │
│                                                              │
│  Stage 1: Pattern Detection ─── Regex for requirement IDs,   │
│           (Deterministic)        numeric thresholds, units   │
│                                                              │
│  Stage 2: LLM Structured ────── AI extracts semantic fields  │
│           Extraction             into typed JSON schemas     │
│                                                              │
│  Stage 3: Validation ─────────── Schema enforcement, unit    │
│           & Normalization        normalization, deduplication │
└──────────────────────────────────────────────────────────────┘
                                │
                  ┌─────────────┴─────────────┐
                  ▼                           ▼
┌──────────────────────────┐   ┌──────────────────────────┐
│   MULTI-SIGNAL LINKING   │   │  DETERMINISTIC RULES ENGINE │
│                          │   │                          │
│ · Explicit ID matching   │   │ · Missing verification   │
│ · Keyword overlap        │   │ · Threshold mismatches   │
│ · Semantic similarity    │   │ · Unit conflicts         │
│ · Category relevance     │   │ · Pass/fail contradictions │
│ · Composite confidence   │   │ · Orphan tests           │
│   score (0.0 – 1.0)      │   │ · Duplicate requirements │
└──────────────────────────┘   └──────────────────────────┘
                  └─────────────┬─────────────┘
                                ▼
┌──────────────────────────────────────────────────────────────┐
│                       HUMAN REVIEW & EXPORT                  │
│                                                              │
│  · Review suggested links with evidence & confidence scores  │
│  · Approve / reject / manually create trace links            │
│  · Investigate flagged issues with source provenance         │
│  · Export verification matrix & missing evidence reports     │
└──────────────────────────────────────────────────────────────┘
```

Screenshots

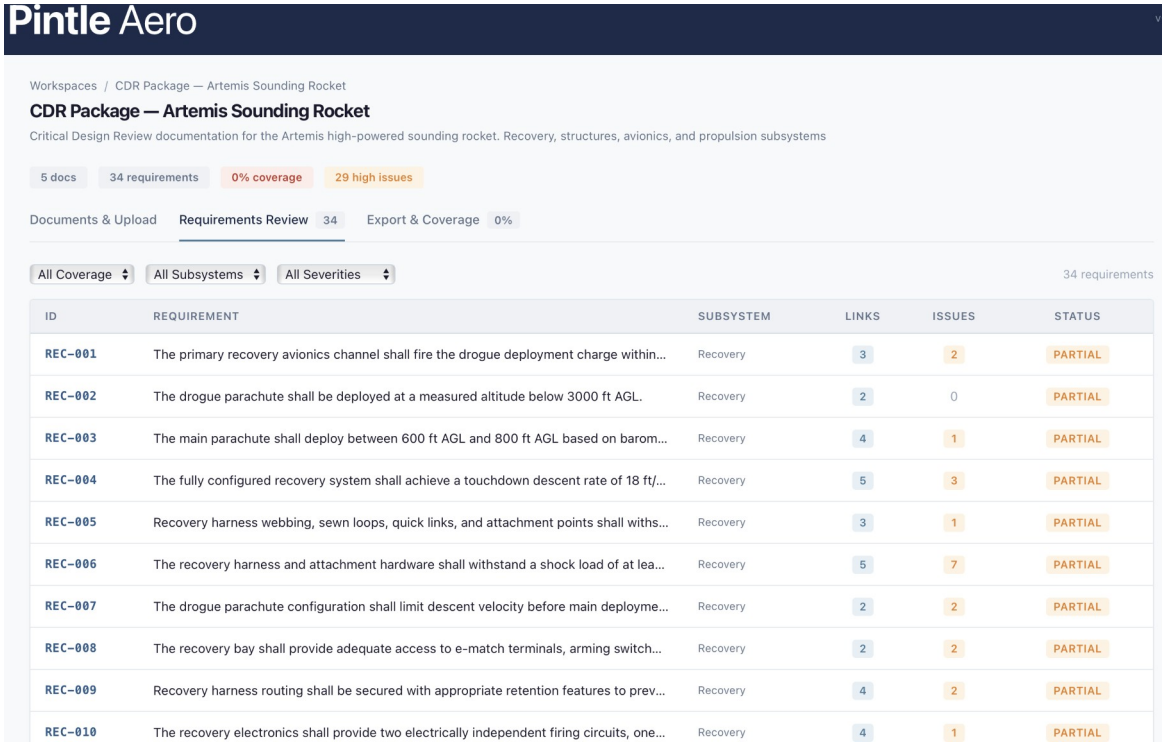
Workspace & Document Upload





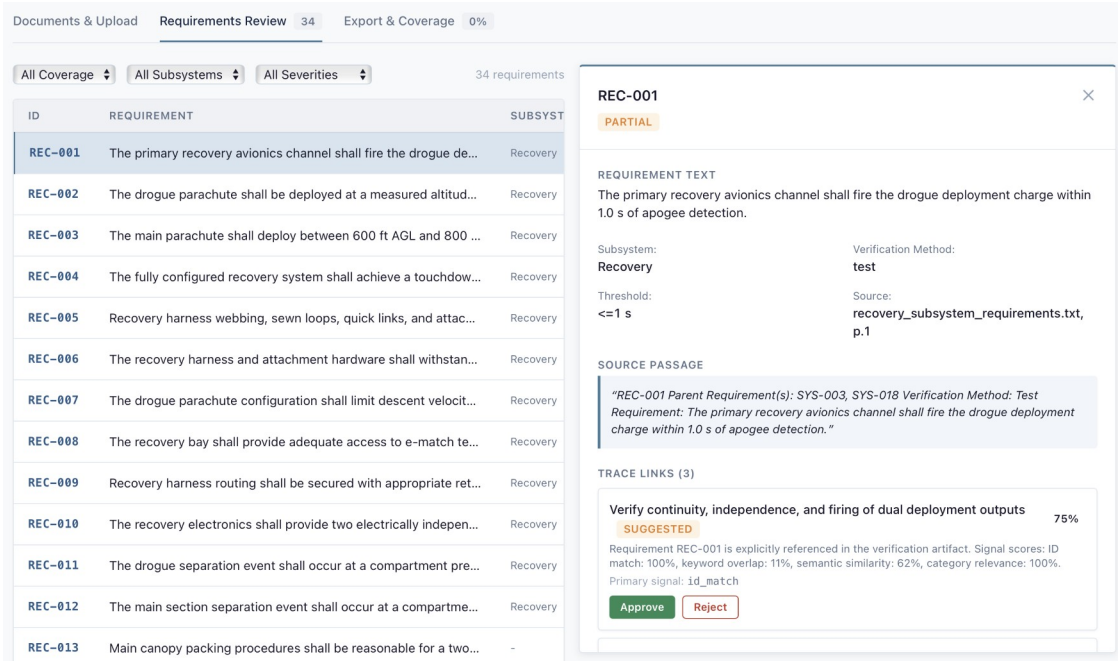
Upload engineering artifacts, assign document categories, and launch analysis.

Requirements Review — The Power Screen



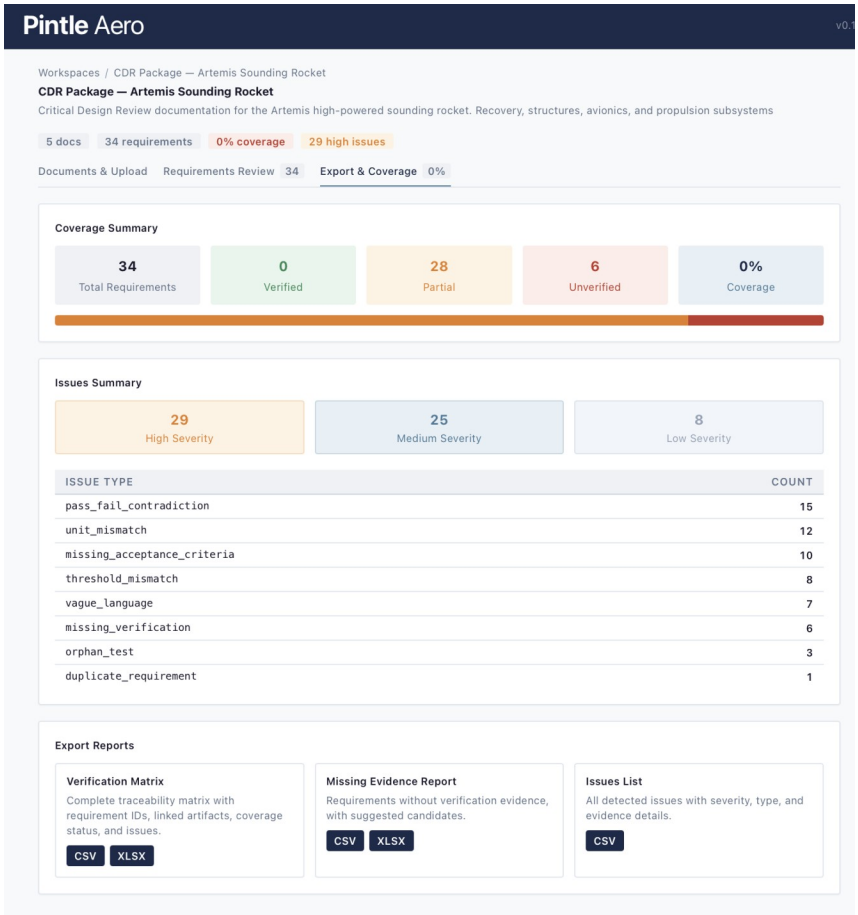
Browse extracted requirements with linked artifacts, confidence scores, and issue flags. Expand any row to see full provenance and side-by-side evidence.

Issue Investigation



Drill into threshold mismatches, unit conflicts, and pass/fail contradictions with exact source evidence from both documents.

Export & Coverage Summary



View coverage metrics at a glance and export verification matrices and missing evidence reports for design reviews.

Key Features

Structured Extraction, Not Chat

Pintle Aero does not ask an LLM open-ended questions about the documents. It extracts **typed entities** — requirements with IDs, thresholds, units, acceptance criteria; test artifacts with measured values, outcomes, and referenced requirement IDs — all conforming to defined schemas with mandatory provenance.

Multi-Signal Link Suggestion

The trace link engine fuses four signals into a composite confidence score:

Signal	Weight	Method
Explicit ID Match	0.50	Requirement ID found verbatim in artifact text
Keyword Overlap	0.20	Jaccard similarity of domain nouns after normalization
Semantic Similarity	0.20	Cosine similarity of embedding vectors
Category Relevance	0.10	Bonus for complementary document categories

Every suggestion includes its confidence score, the primary signal that drove it, and a natural-language explanation.

Deterministic Rules Engine

Safety-critical checks are **never delegated to an LLM**. Threshold mismatches, unit conflicts, and pass/fail contradictions are detected by a rules engine operating on structured data with full unit normalization (via Python's `pint` library). This eliminates hallucination risk for the checks that matter most.

Provenance-First Architecture

Every extracted field, every suggested link, and every detected issue traces back to a specific page and passage in a source document. The user can always ask "show me where this came from" and get an answer in one click.

Quick Scan Mode

Upload a single requirements document — no test artifacts needed — and get an instant quality report flagging vague language, missing acceptance criteria, absent units, and unstated verification methods. Value in under 60 seconds.

Human-in-the-Loop Review

Suggested links must be approved or rejected by the engineer. Issues can be overridden or marked as accepted risk. Pintle Aero is a copilot, not an autopilot.

Technology Stack

Layer	Technology
Frontend	React / Next.js, TypeScript, Tailwind CSS
Backend	Python, FastAPI
AI / Extraction	Claude API (Structured Outputs), Sentence Transformers
Rules Engine	Python deterministic checks, `pint` unit normalization
Document Parsing	PyMuPDF (PDF), python-docx (DOCX), openpyxl (XLSX)
Persistence	SQLite (entities, links, issues, embeddings)
Deployment	Docker Compose — single *`docker compose up`* command

Architecture

Pintle Aero uses a modular service architecture with clear separation between AI-powered extraction and deterministic validation:

```
┌──────────────────────────────────────────────────────────────────────┐
│                        FRONTEND (React / Next.js)                    │
│  ┌──────────────────┐  ┌──────────────────────┐  ┌──────────────────┐
│  │ Workspace &      │  │ Requirements         │  │ Export &         │
│  │ Upload           │  │ Review & Issues      │  │ Coverage         │
│  └──────────────────┘  └──────────────────────┘  └──────────────────┘
└──────────────────────────────────────────────────────────────────────┘
                                │ REST API (16 endpoints)
                                ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        BACKEND (Python FastAPI)                      │
│                                                                      │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────────┐
│  │ Document         │  │ AI Extraction    │  │ Rules Engine         │
│  │ Processing       │  │ Service          │  │ (Deterministic)      │
│  └──────────────────┘  └──────────────────┘  └──────────────────────┘
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────────┐
│  │ Linking          │  │ Reporting &      │  │ Quick Scan           │
│  │ Engine           │  │ Export           │  │ Service              │
│  └──────────────────┘  └──────────────────┘  └──────────────────────┘
└──────────────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────────────┐
│  SQLite (entities, links, issues, embeddings)
│  Local File Storage (uploaded documents)
└──────────────────────────────────────────────────────────────────────┘
```

Issue Detection Capabilities

Issue Type	Severity	What It Catches
Missing Verification	High	Requirement has no linked test evidence
Threshold Mismatch	High	Measured value falls outside requirement's acceptance range
Pass/Fail Contradiction	High	Report says "PASS" but measured value violates criteria
Unit Mismatch	Medium	Linked artifacts use incompatible units (e.g., lbf vs. N)
Orphan Test	Medium	Test report exists but maps to no requirement
Duplicate Requirement	Low	Two requirements are near-identical in the same subsystem
Vague Language	Low	Requirement uses unmeasurable terms ("adequate", "sufficient")
Missing Criteria	Medium	Requirement has a threshold but no measurable acceptance criteria

---

Demo Dataset

Pintle Aero ships with a sample dataset based on a **high-powered sounding rocket project**, including system-level requirements, recovery and structures subsystem specs, test procedures, and test reports. The dataset includes intentionally planted issues — missing verification evidence, threshold mismatches, unit conflicts, and pass/fail contradictions — to demonstrate the tool's detection capabilities.

Who This Is For

- **Verification & validation engineers** building traceability matrices for design reviews
- **Systems engineers** preparing compliance packages for CDR, TRR, or FRR milestones
- **Student aerospace design teams** (NASA Student Launch, AIAA DBF, CubeSat programs) who need to prove verification rigor
- **Engineering managers** who need a quick answer to "are we ready for this review?"

What This Is Not

Pintle Aero is a decision-support assistant. It does **not**:

- Replace formal systems engineering tools (DOORS, Cameo, Polarion)
- Autonomously approve compliance or sign off on verification
- Serve as a certification authority
- Replace human engineering judgment

It catches problems early so humans can fix them before they matter.

Future Roadmap

The following capabilities are planned for future releases:

- Interactive traceability graph visualization
- DOORS / Jira / Polarion import and export adapters
- Document version comparison and change impact analysis
- Local LLM deployment for air-gapped environments
- Batch analysis API for CI/CD integration
- Multi-user collaboration with role-based access

About the Builder

Anjana Gireesan Junior, Aerospace Engineering — Purdue University

Built with hands-on experience from NASA Student Launch, where incomplete traceability during design reviews was a recurring pain point. Pintle Aero was born from the question: *"Why are we still doing this in spreadsheets?"*

Contact

For a live demo, source code access, or to discuss the project:

- **GitHub:** https://github.com/agireesa15/pintle-aero-ext
- **LinkedIn:** linkedin.com/in/anjanagireesan